面向 CODESYS 平台的 ST 代码自动生成系统

——GenAI 课程大作业

沈彭宇 张亚彬 习博文

浙江大学

2026 年 11 月 11 日

目录

1. 问题总览

2. 数据与规约

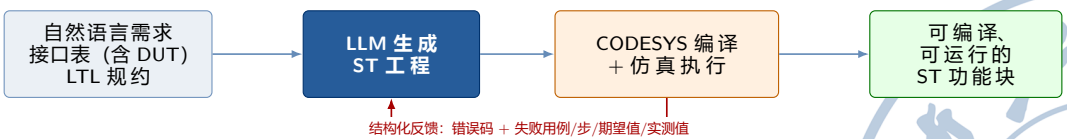
3. Agent 框架

4. 结果与解释

5. 总结



问题总览



难点不在“像不像”

- 语法与工程：DUT、Standard 库、调用图
- 行为语义：状态机、定时器、边沿、优先级
- 评测可信：必须**真实执行**

什么算“做对了”

编译通过 **且**用例在仿真中逐拍对拍通过；
用例对模型不可见

交付

30 题数据与规约（75 条 LTL / 54 组用例）
自研 harness · CODESYS 编译 + 仿真服务
统一评测管线

四种设置

{Codex CLI, 自研} × {DeepSeek, Qwen}
各 30 题，统一 5 轮修复预算，无工具故障

数据集

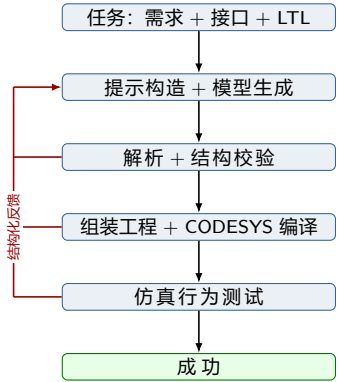
维度	数量
总题数	30（公共 20 + 自建 10）
难度	easy 9 / medium 12 / hard 9
形式化规约	75 条（2.5 条/题）
测试用例	54 组、199 个扫描步

每题包含：`FUNCTION_BLOCK` 接口（含 DUT）+ 中英需求 + LTL + 行为用例 + 可编译参考实现

- **自建题**：交通灯、水箱、自动门、分拣、主备泵、报警闭锁、星三角、停车场、恒压供水、电梯等 PLC 场景

- **多样性**：离散制造/流程工业/电机/安全/队列/字符串；含 DUT 3 题、`VAR_IN_OUT` 4 题
- **规约**：互斥、响应、保持、范围不变量、错误码
- **用例**：正常流程、边界、优先级冲突、错误处理；**每步写明测试目的**并关联到 LTL 规约
- **自检**：参考实现 **30/30** 编译且 **30/30** 通过自己的用例

架构：单 Agent 反馈闭环



- 一次生成完整工程，每轮整体重写，不做局部补丁
- **结构校验前置**：格式/接口错误不消耗 CODESYS 冷启动
- **上下文**：仅保留最近 2 轮；错误按 (code, text) 去重
- **终止**：通过 / 5 轮 / 代码无变化 / 失败签名重复 / 工具故障
- 测试用例**对模型不可见**，仿真基于 CODESYS 官方运行时

实验设置

	deepseek-v4.1-flash-1017	qwen3.7-flash
现有框架（Codex CLI）	实验 1	实验 2
自研 harness	实验 3	实验 4

- Codex CLI 包装为与自研相同的 `chat(messages)` 客户端：**提示、反馈文本、轮次上限、编译测试管线完全一致**
- 差异只在生成组织方式（Codex 自带 agent 循环与工具上下文），因此结果差异只能归因于框架本身
- **指标**：首次编译、最终编译、测试通过率、token、墙钟时间、轮次

总体结果：DeepSeek 打平，Qwen 拉开差距

设置	首次编译	最终编译	测试通过	平均轮次	平均耗时
自研 + DeepSeek	96.7%	100.0%	**83.3%**	1.77	163 s
自研 + Qwen	76.7%	93.3%	**66.7%**	2.53	295 s
Codex + DeepSeek	100.0%	100.0%	**83.3%**	1.70	342 s
Codex + Qwen	30.0%	83.3%	**46.7%**	3.53	315 s

83.3% / 83.3%

DeepSeek 下两框架测试通过率**完全持平**

66.7% vs 46.7%

Qwen 下自研**高出 20 个百分点**

96.7% vs 100%

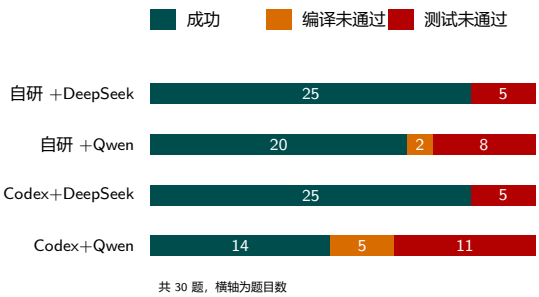
首轮编译：自研靠反馈追平最终结果

代价：同样的通过率，输入成本差一个量级

设置	输入 tokens	输出 tokens	平均耗时	相对 Codex
自研 + DeepSeek	75,058	383,312	163 s	输入 **13%**、耗时 **48%**
自研 + Qwen	131,255	503,606	295 s	输入 **9%**、耗时 **94%**
Codex + DeepSeek	565,110	521,147	342 s	—
Codex + Qwen	1,404,958	394,781	315 s	—

- 通过率相同（83.3%）的前提下，自研输入 token 为 Codex 的 **1/7**、耗时约 **1/2**
- 差距几乎全在**输入侧**：Codex 的 agent 循环反复累加系统提示、工具定义与历史
- Qwen 下自研耗时与 Codex 相当（94%），但通过率高出 20 个百分点
- 输出 token 两框架同量级（383k vs 521k），说明“想的量”相当，差的是“看的量”

解释一：差距落在编译层，不是行为层



- DeepSeek 两种框架**编译失败均为 0**：编译闸门已不是瓶颈，扣分全在行为层
- 自研 + Qwen 只剩 2 题编不过，靠反馈把 28 题送进测试
- Codex + Qwen 有 **5 题最终仍编译失败**：首轮编译 30%，5 轮预算常耗在语法上
- 只看编译通过的题，行为通过率为 83.3% / 83.3% / 71.4% / 56.0%：Qwen 下两框架的差距从 **20 个百分点收窄到 15 个百分点**，其余差距来自没编译过的题

解释二：Qwen 下差距从哪来

指标（Qwen）	自研	Codex
首轮进入编译	**30/30**	**21/30**
接口预校验拦下	0	7
块解析拦下	0	2
每轮编译错误条目	**1.17**	**2.66**
首轮错误数（中位/均值）	0 / 1.2	2 / 3.7
首轮编译通过	23/30	9/30
双方都过的题平均轮次	**1.38**	**2.00**

失败题集高度重合——差距在于**同等能力下 Codex 要更多轮才收敛**（上限 5 轮）.

- **输出结构不成立**：Codex 侧把 `REAL/BOOL` 当 DUT 名、数组别名写成 `kind=array`、整段没有 POU 块——9 题的提交根本没走到编译器
- **错误更像通用语言**：C189 74、C9 68 的样本是 `应该用',' 替换')'、应该用表达式代替'!'` ——把 ST 写成 C/JS 风格（`!`、`if(...)`、`//`）
- **提示层稀释**：Codex 以通用 agent 身份运行，模型要同时满足“我是编码代理”与“输出严格块格式的单个 FB”；Qwen 指令跟随弱，冲突时滑向通用写法——这正是 DeepSeek 打平、Qwen 拉开的原因
- **上下文层**：中位输入 token 11,314 vs 1,573（7.2 倍），输出预算被压缩（3,354 vs 6,101）

总结

1. 建成 30 题数据集，逐题 LTL 规约、测试用例（含逐步测试目的）与参考实现；参考实现 30/30 通过编译与仿真，形成可复现基线
2. 自研 harness 在 DeepSeek 下达到与 Codex 相同的测试通过率（83.3%），输入 token 约为其 1/7、耗时约 1/2；在 Qwen 下仍更优（66.7% vs 46.7%）
3. 差距主要落在**首轮进入编译的比例**（30/30 vs 21/30）、每轮错误密度（1.17 vs 2.66）与上下文成本，而非“模型想得不够”；剩余失败集中在行为语义

局限性

- 规约未接入 PLCVerif（选做项，未做）；每题仅采样 1 次，Qwen 下两框架差异 Fisher 精确检验 $p = 0.19$，未达显著
- 12 个扫描步无断言、34 个布尔输入从未置 TRUE，通过率是“现有用例下”的通过率
- 下一步：接入 PLCVerif、补强用例覆盖、多平台验证

把“能不能编译、跑得对不对”做成结构化反馈，弱模型也能被推上来；但没有对所有模型都更好的框架.